FIT5226 Project

Stage 2 - Multi-agent coordination

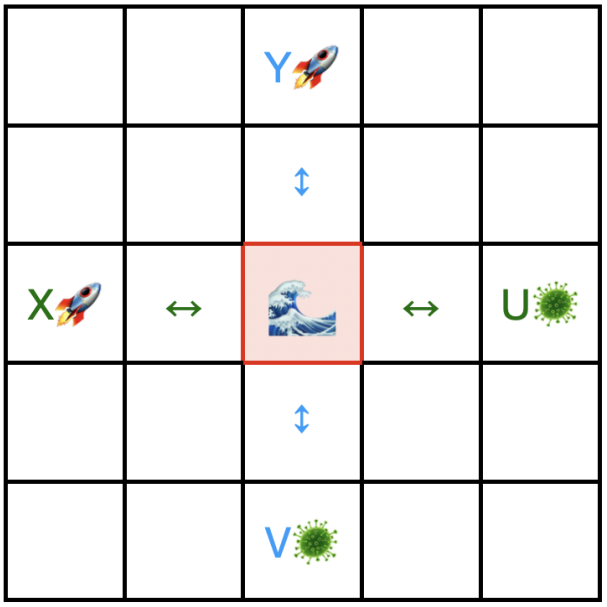
This document describes Stage 2 of the FIT 5226 project, the final stage. We build on the work done in Stage 1. If you didn't manage to solve Stage 1, a Master solution will be made available on the Moodle site for you to use as a foundation.

Assessment	Topic	Release	Due	Weight	Assessment mode
in-semester	Project Stage 1 (Table-based RL, single agent)	Wk3	April 11	15%	Individual
in-semester	Project Stage 2 (MARL coordination task)	Wk 6	May 15	35%	Individual
exam	All topics but no programming			50%	

This project constitutes your *entire* in-semester assessment for the unit.

Stage 2 (the current Phase) concerns *Multi-agent coordination*.

Scenario for Stage 2



You will write code for multiple agents of two different types to perform q-learning for a coordination task in a small grid world. The emphasis in this assignment is not on the coding itself—this should be straightforward after Stage 1. The emphasis is not even on successfully learning all aspects of all tasks. Your agents may or may not be able to learn the more advanced aspects of the scenario but this is not what determines success in this assignment. The emphasis is on understanding how RL behaves in this scenario and what makes one variant of the problem harder than the other one. For a high level of achievement we expect you to systematically use structured experimentation to find the answer to this and/or to be able to explain this in a theoretically grounded framework.

Here is the scenario. *Read it carefully, the details are important!*

The space agency has contracted your company to design autonomous robots for sample collection in rugged terrain on a remote planet.

The mission plan designates four relevant locations: The landing sites of two spaceships (X in the west, Y in the north) and two target sites (U in the east, V in the south), where we hope to find evidence of life. The robots will be deployed from the spaceships at X and Y and their task is to travel to U and V, retrieve soil samples and come back to the same ship they were deployed from. Robots deployed at X have to sample at U and robots deployed at Y have to sample at V. The locations of X, Y, U and V are fixed as shown in the diagram.

We don't know much about the planet except (1) that the terrain is dangerous and movement is extremely difficult, so that it is important that all routes have to be kept as short as possible; (2) there is a mysterious playa lake (an ephemeral lake) at the point where the shortest connections between X and U and Y and V, respectively, intersect. This lake floods and dries out at surprisingly high speed (but this may not be so strange given that this is a very strange planet anyway).

As the head of the engineering department, you decide to build swarms of tiny robots, so that an almost *continuous stream of robots* can run between each ship and its target sampling location.

Your project experiences two phases as another department meddles in it.

Phase 1:

Robert, the head of the cost-cutting department wants your job, so he tries to sabotage your department by deciding that only half of the robots can be built with reliable waterproofing. The robots deployed at X (let's call them Type A) only get a cheap and somewhat dodgy waterproofing so that there is a serious risk of damage if they attempt to cross the lake when it is flooded. The ones to be deployed at Y (let's call them Type B) get full waterproofing and they can cross the lake safely even if it is flooded. At least Robert allocates some budget equipping each robot with a water sensor that indicates whether the lake is flooded.

Robots get deployed at random times, so that you have a more or less continuous stream of robots going between X and U (Y and V, respectively). Importantly, water damage is not the only danger on the lake. Since it is also the crossing of the shortest paths for the two robot types and, there is a high danger of collision between robots of different types, which would seriously damage them (robots of the same type are assumed not to collide; they are built to evade each other).

Luckily, Sarah, your smartest engineer, saves the project by advising that she can equip the robots with reinforcement learning so that type A can learn to adjust its crossing behaviour according to the risk of water damage and both types can learn to avoid collisions.

Phase 2

Realising that he won't succeed in sabotaging your project because of Sarah's skills, Robert changes tack and advises that the space agency doesn't fully trust her method. In an attempt to appear genuine, he now allows you to fully waterproof all robots of both types. Since there is no danger of water damage anymore, Robert decrees that the robots must no longer be penalised for entering the flooded lake so that they can figure out the optimal behaviour. However, the collision danger in the lake remains.

Robert declares that your reinforcement learning should still be able to avoid collisions by learning that one kind of robot should only cross when the lake is dry and the other kind should only cross when the lake is flooded. In some sense they should learn to use the lake as a traffic light. For reasons that remain unclear, you are not allowed to program their behaviour explicitly in this way but they have to learn this on the job.

Robert cunningly argues that this should indeed be quite simple since, as he says, "using the lake like a traffic light is just a different equilibrium of the collective behaviour so they should learn this."

Sarah warns you that this will be extremely difficult to achieve using q-learning and that she has serious doubts the project will succeed under these conditions.

Tasks

Note that you don't have to address all tasks listed below for this assignment. For details which tasks have to be solved for which grade level, please refer to the indicative rubric at the end of this document.

As the head of engineering, it is your responsibility to figure out how the project can be conducted successfully. Is Sarah right? Can you outsmart Robert?

You decide to build a small simulation to support your decision making. You set up a 5x5 grid-world as in the diagram and populate it with the two agent types, A and B. Both types have the same five actions that they can execute, namely to step to any neighboring field in the four cardinal directions (north, south, east, west) or to wait in the current position without moving. Type A starts at X and when it reaches location U it *automatically* picks up a sample. When it has returned to X it *automatically* discharges the sample. At this point, it has completed its task. Likewise, type B, deployed from Y, automatically picks up a sample when it reaches V and automatically discharges the sample when it returns to Y. No specific action is required from a robot for picking up or dropping an item, it only needs to step into the corresponding location.

The task each robot has to learn is to find the shortest way to their designated sampling location and to avoid collisions and water damage. Each robot can observe its own location, whether it carries a sample, and the binary state of the lake (dry/flooded) but not the other agents' locations.

Task 1:

Sarah's theory about Phase 1 is correct. The robots can learn that Type A should only cross when the lake is dry and Type B consequently should only enter the lake when it is flooded so that the crossing is guaranteed to be collision-free (as there will be no Type A in the lake at that time).

Build a simulation to demonstrate that the agents will learn this behaviour with q-learning.

At each time all agents step simultaneously. This means that an agent sees the results of the actions by all other agents in the same time step. For example, if two agents stand next to the lake at time *t* and both decide to step into the lake, they will collide in time step *t*. Agents only see the final result of these steps, so if an A agent is in location X at time t and a B agent moves to X at t+1 while the A agent moves away from X at t+1 they do not collide.

The state of the lake is updated in-between successive time steps according to a probability *p* that determines the probability of it changing its state from dry to flooded (or vice versa) at this moment.

The exact sequence of steps is the following:

1. all agents observe the state

2. all agents choose their actions based on this state
3. all agents execute their actions
4. the environment changes its state in response to the actions (but no lake change)
5. the reward is evaluated
6. the lake may change its state (probabilistically)

Let's consider an example: Agent x is to the left of the lake and the lake is dry. x decides to step right into the lake, receiving the normal step penalty. Now the lake floods (starts to flood) and the agent decides to again step right so that x departs from the lake. Since x is out of the lake as a result of its action, it will getting wet stepping into water will only be given if x steps into or does not depart from the flooded lake against better knowledge.

Your risk-assessment engineers advise that the following rewards adequately reflect the cost and likelihood of damage. You may modify these but they provide a good starting point:

Step = -5, Wait = -3, Collision = -20, Type A entering water = -20

Of course, these scale with the other rewards and the above is valid for sample-location-reached = 10, sample-delivered = 50.

<u>Hints</u>:

1. We strongly advise using tabular Q-Learning and not Deep-Q. While Deep-Q will work, it is absolute overkill, can get quite messy to set up correctly, and it blows the learning time out of proportion.
2. As in the first assignment, no additional heuristics or trickery with the rewards (encoding heuristics in the reward schema) are permissible.
3. Learning this task is quite sensitive to the setting of the discount factor γ.
4. When your simulation at first does not succeed, Sarah advises that it is very important that you account for both possible states of the lake in the next state. It could dry out or flood at any point of time. Make sure to follow her advice.
5. You are allowed to split the simulation into multiple learning phases instead of just a single integrated simulation in which both types learn in parallel. However, you may only do so if you can provide a sound argument why this is equivalent to letting them learn in an integrated way and on-the-job.
6. Please read the section about generative AI use at the end of this document very carefully. While you are allowed and even encouraged to use AI for coding and learning purposes, you must be able to explain the submitted product completely yourself. This applies to all tasks.

Task 2:

Build a simulation to investigate whether Sarah or Robert is right regarding Phase 2.
Find out what happens in the changed scenario and analyse the outcomes.

Task 3:

Back up your insights and observations from Task 1 and Task 2 from a game-theoretic perspective. Reduce the whole scenario to the core crossing decision (i.e. assume that the robots already know how to navigate the shortest path). Devise a game that reflects this situation meaningfully and use replicator dynamics to find out what the behaviour converges on. You are free to simulate replicator dynamics numerically or to solve it analytically as you see fit.

Task 4 (max 150 words)

Explain your findings from the previous tasks from a theoretical perspective. In other words, give a conceptual argument that is firmly grounded in the theory of reinforcement learning and game theory that explains why the outcomes observed in Task 1-3 make sense.

Submission Instructions

Submission will be via the Moodle platform. Detailed submission instructions will be published on Moodle in the Assignment section.

Use of Generative AI

You are allowed and, in fact, encouraged to use Generative AI to solve your assignment. If you decide to do so, you must treat the AI like another external author (as a non-authoritative author whom you mistrust, given how much content is made up by Chat GPT and similar AIs). It is entirely your own responsibility that the content of your submission is correct and you can only use generated content to the extent that you could use materials provided by an external author. The AI is not part of your project team. If you use code or insights that an AI provided you must be able to fully explain every detail of it.

If you submit code that you are unable to explain in an interview, this will attract zero marks.

You must give a declaration that fully explains how and for which components you used generative AI. If you have not used AI, you must still include a declaration stating that no AI was used.

Any use of generative AI must be appropriately acknowledged (see Learn HQ).

Indicative Rubric

Your solution will be marked on

- Correctness but not elegance of the implementation
- Design of the evaluation metrics
- Correct and insightful interpretation of the outcomes
- Clarity of the discussion and explanations
- Well-designed experimentation where used and/or required
- Level of linking observations meaningfully to theory

Pass Level (Task 1)

You have implemented q-learning correctly for at least the first phase (Task 1). Your implementation clearly learns (but maybe not to optimality). You have defined meaningful metrics and demonstrated the learning using these metrics. Please familiarise yourself with the requirement to be able to fully explain all codes and outputs as outlined under “Use of Generative AI” above.

Credit Level (Task 1 & 2)

You have implemented both phases correctly (minor errors may be present) and your implementation learns the first case to optimality (Task 1 and Task 2). Convincing evidence for full learning is given in the first case using the metrics defined. The final policies learned for both cases have been visualised, correctly interpreted and compared. The relationship between the solution qualities in both phases has been explained in terms of the policies learned.

Distinction Level (Task 1-3)

Requirements for credit *plus either of the following*

(1) A solid attempt has been made at using GT/EGT to back up the reasoning presented (Task 3). This attempt shows a good understanding of GT/EGT and its relation to RL, relates meaningfully to the problem and is mostly complete but it may contain minor mistakes and/or omissions.

Or

(2) At least one meaningful experiment has been designed, conducted and discussed to analyse the harder nature of Phase 2. The discussion shows a good understanding of the harder nature of the second problem. We normally expect such experiments to proceed by modifying a particular structural aspect of the problem to reveal how this aspect impacts the learning. The experiment must be designed and outcomes of the experiment are meaningfully explained and analysed.

    One interesting variation to investigate is : How do different levels of step penalties influence the outcome in Phase 2?

You are allowed to use this suggestion but you are free to design other variations if they reveal important aspects of the behaviour.

High Distinction Level (Task 1-4)

Both phases are implemented correctly and compared as simulations. At least the simulation of the first phase learns optimal behaviour. The simulation outcomes are observed correctly and documented using the metrics defined.

The difference between the hardness of the problems for RL is explained with arguments that are solidly grounded in theoretical concepts of Reinforcement Learning and Game Theory (Task 4). They show a comprehensive understanding of the different characteristics of the problems for RL and their relation to game theory.

Theoretical explanations must be correct but only need to be given conceptually in broad sketch terms, i.e. they may but do not have to contain detailed mathematical working out.

Don’t let Robert win! Ideally, you have found a way to outsmart Robert, maybe with Sarah’s help, but this is not a part of the requirements for a HD.